

claude-windows / 902e27a6-e026-4b55-8c26-f2c47329cc44

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\subagents\workflows\wf\_c287c071-8e9\agent-ac549cfab257e5c64.jsonl`
- SHA-256: `f50bedc3cec6229830e692d25fb23973c08b5b9e1434f943b48814e94993d898`
- Source modified: `2026-07-07T14:56:05+00:00`
- Imported at: `2026-07-08T16:00:02+00:00`
- project: `wf\_c287c071-8e9`
- session_id: `902e27a6-e026-4b55-8c26-f2c47329cc44`

Transcript

1. user (2026-07-07T14:50:27.842Z)

You are reviewing an uncommitted change in a git worktree at:
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval
Branch: fix/worker-skip-revalidation-on-dispatch (based on origin/master).

THE BUG BEING FIXED: On the cloud-serving path, the control-plane (backend/orchestration.py) runs validators and ONLY dispatches to the Cloud Run worker on PASS. The worker (backend/worker/__main__.py cmd_infer) then REDUNDANTLY re-ran validation with its OWN config (different min_blur_threshold), rejecting a clip the control-plane already passed. Live failure 2026-07-07: passed at min_blur_threshold=8.0 on the control-plane, worker rejected at its default 20.0 ? GPU dispatch succeeded but the CUJ failed with worker rc=2.

THE FIX (skip-on-dispatch): a new ComputeJobSpec.skip_validation bool (default False) ? backend/compute/cloud_run.py emits a "--skip-validation" worker CLI flag when set ? backend/worker/__main__.py cmd_infer honors it (skips validator_registry.run_all_with_context, sets val_results=[], val_context={}) ? backend/orchestration.py::_maybe_dispatch_remote sets skip_validation=True on every dispatch. The direct "worker infer" CLI (no control-plane) still validates by default. New tests in tests/test_worker.py and tests/test_api_cloud_dispatch.py.

Inspect the ACTUAL code ? do NOT trust this summary. Run: cd "C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval" && git --no-pager diff and read the surrounding functions (not just the diff hunks). The full test suite already passes (1779 passed), ruff + mypy are clean, coverage 85.42%. So do NOT report style nits or things the gate already covers. Hunt for REAL correctness/rollout/consumer defects the tests would miss. Only report a finding you can tie to a concrete failure scenario in THIS repo's code.

A reviewer (lens: rollout) raised this finding about the change. ADVERSARIALLY verify it against the REAL code in C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval (read the files; run git grep / git diff). Default to REFUTED unless you can demonstrate the concrete failure in THIS repo's code. If real, give the minimal fix.

FINDING:
severity=major category=rollout-skew
file=backend/compute/cloud_run.py line=202
summary=The new `--skip-validation` worker CLI flag is emitted on every cloud dispatch, but an older worker Job image that lacks the argparse arg aborts (rc 2) BEFORE writing the done-marker, causing a 30-minute bucket-poll hang then an rc-4 failure on every dispatch during a control-plane-ahead-of-worker rollout window.
detail=The control-plane (Cloud Run Service) and worker (Cloud Run Job) are the same image but SEPARATE resources with independently-pinned digests; README deploy/cloudrun/README.md §2a

already warns they can skew. Unlike #507/#509 which forwarded `decode_backend` via the PRE-EXISTING `--set k=v` arg (orchestration.py:564 ? never argparse-aborts)`, this change adds a genuinely NEW `argparse` flag: `worker __main__.py:743 (`add_argument("--skip-validation")`), consumed by strict `parse_args` at __main__.py:822. orchestration.py:582 sets skip_validation=True on EVERY dispatch, and cloud_run.py:202 appends --skip-validation` to the container argv. If the control-plane is updated to the new image but the worker Job still points at an older digest (the natural 'redploy the service, forget/lag the `jobs update`' sequence ? #510's runlog shows the control-plane being redeployed on its own), the old worker runs `python3 -m backend.worker infer ... --skip-validation` (ENTRYPOINT is bare `python3 -m backend.worker`, no wrapper) and argparse calls SystemExit(2) inside main() BEFORE cmd_infer/_fail/_write_done_marker execute. No done-marker is written. The dispatcher (cloud_run.py run_infer) only bucket-polls the marker (never the Cloud Run API status), so poll_until runs the full ExecutionPolicy.max_wait_sec=1800s (30 min), then PolicyTimeout -> orchestration.py:593-595 surfaces a generic 'cloud dispatch error' (rc 4). This affects EVERY dispatch, including clips the control-plane PASSED ? a strict regression versus the pre-fix fast rc-2, and the failure is opaque (the real 'unrecognized arguments: --skip-validation' only appears in the Job execution log, not to the control-plane). This is stricter coupling than #507's decode_backend yet ships NO deploy-together release note (README §2a got one for #507) and no code guard. Correct mitigation: (a) a README §2a-style 'roll worker image + control plane together, worker-first' note, since a worker-side parse_known_args guard added now cannot retroactively fix the already-deployed old image; reverse skew (new worker, old control-plane) is safe ? the old control-plane simply never sends the flag. Separately verified NON-issue: adding skip_validation as a trailing `bool = False` field to the frozen ComputeJobSpec is benign ? bool is hashable, all `ComputeJobSpec(` call sites use kwargs, and the spec is in-memory only (not pickled/persisted across versions).`

`failure_scenario=Operator` builds the new image, redeploys the rally-control-plane Service to it, but the rally-worker Job still pins the prior digest (its ``jobs update` is lagged or forgotten). A user uploads a clip; the control-plane validates it PASS at min_blur_threshold=8.0 and dispatches with --skip-validation. The old worker Job argparse-aborts rc 2 with no done-marker; the control-plane bucket-polls for 30 minutes, then fails the job with 'cloud dispatch error' (rc 4). Every dispatch in the skew window behaves this way, and the underlying cause (an unrecognized CLI arg) is invisible from the control-plane surface.`

2. assistant (2026-07-07T14:50:31.634Z)

I'll investigate this finding against the real code. Let me start by examining the diff and the relevant files.

3. user (2026-07-07T14:50:33.952Z)

```
diff --git a/backend/compute/base.py b/backend/compute/base.py
index 279606e..31af3fe 100644
--- a/backend/compute/base.py
+++ b/backend/compute/base.py
@@ -28,6 +28,13 @@ class ComputeJobSpec:
     segmenter: Optional[str] = None
     # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
     frozen/hashable.
     config_overrides: tuple[str, ...] = ()
+    # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving control-
+    plane) is
+    # the authoritative validation gate: it runs the SAME validators with ITS resolved config
+    and only
+    # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video decode,
+    and
+    # (c) outright WRONG when the worker's config resolves different thresholds ? it can REJECT
+    a clip
+    # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no control-
+    plane)
+    # validating exactly as before.
+    skip_validation: bool = False

@@dataclass(frozen=True)
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
```

```

index 841785d..7247ace 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -196,6 +196,10 @@ class CloudRunCompute(ComputeBackend):
    ]
    if spec.segmenter:
        argv += ["--segmenter", spec.segmenter]
+
+   if spec.skip_validation:
+       # The control-plane already validated + only dispatches on PASS ? tell the worker
not to
+       # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
+       argv.append("--skip-validation")
    for kv in (*spec.config_overrides, *self._container_overrides):
        argv += ["--set", kv]

diff --git a/backend/orchestration.py b/backend/orchestration.py
index 5e9f510..79b4fed 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -568,11 +568,18 @@ def _maybe_dispatch_remote(
    prefetch = getattr(wasb_cfg, "prefetch_batches", None)
    if prefetch:
        overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
+
+   # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
+   # gate ? it already ran the validators with its resolved config just above (only a PASS
reaches
+   # here; a failure returned "failed" before dispatch). A worker re-run would decode the
video a
+   # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
+   # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
+   # then worker-rejected at its default 20.0 after a successful GPU dispatch).
    spec = ComputeJobSpec(
        video_uri=req.video_path,
        out_uri=out_root,
        segmenter=seg_name,
        config_overrides=tuple(overrides),
+
+       skip_validation=True,
    )
    try:
        # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 88c50f6..9b19215 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -186,6 +186,7 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
    out_uri=args.out_uri,
    segmenter=args.segmenter,
    config_overrides=tuple(args.set or ()),
+
+   skip_validation=bool(getattr(args, "skip_validation", False)),
    )
    try:
        result = compute.run_infer(spec)
@@ -259,10 +260,25 @@ def cmd_infer(args: argparse.Namespace) -> int:
    # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-
identical).
    video_id = compute_video_id(local_video)

-   # 3. VALIDATE (same registry as the main CLI).
-   val_results, val_context = validator_registry.run_all_with_context(
-       local_video, config
-   )

```

```

+ # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
+ # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate: it runs
the
+ # SAME validators with ITS resolved config and only dispatches on PASS. Re-validating here
is
+ # redundant, decodes the whole video a SECOND time, and is outright WRONG when the worker's
config
+ # resolves different thresholds than the control-plane's ? observed 2026-07-07: a 5312x2988
clip
+ # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0, then the
worker
+ # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the fact.
+ # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote) skips
it; the
+ # direct ``worker infer`` CLI has no control-plane, so it still validates by default.
+ if getattr(args, "skip_validation", False):
+     print(
+         "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
+         "gate and already validated this input."
+     )
+     val_results, val_context = [], {}
+ else:
+     val_results, val_context = validator_registry.run_all_with_context(
+         local_video, config
+     )
+     failures = [r for r in val_results if not r.passed]
+     db = Database(os.path.join(scratch, config.output.db_name))
+     db.add_video(
@@ -724,6 +740,14 @@ def build_parser() -> argparse.ArgumentParser:
+         metavar="k=v",
+         help="config override, e.g. --set indexing.skip_ai_handoff=true",
+     )
+     pi.add_argument(
+         "--skip-validation",
+         action="store_true",
+         help="skip the worker's re-validation: the dispatcher (control-plane) already ran the "
+         "validators with its resolved config and only dispatches on PASS, so re-validating here
is "
+         "redundant, decodes the video again, and can contradict the gate. Default-OFF: the
direct "
+         "``worker infer`` CLI still validates.",
+     )
+     pi.set_defaults(func=cmd_infer)

+     pt = sub.add_parser(
diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
index 8382234..19c37d9 100644
--- a/tests/test_api_cloud_dispatch.py
+++ b/tests/test_api_cloud_dispatch.py
@@ -236,6 +236,22 @@ def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
+     assert "indexing.wasb.prefetch_batches" not in joined

+def test_cloud_dispatch_skips_worker_revalidation(cloud_env):
+    """The control-plane is the AUTHORITATIVE validation gate: it validated + only dispatches
on PASS
+    (registry.run_all_with_context is stubbed to pass in cloud_env). Re-validating on the
worker is
+    redundant, decodes the video a second time, and ? because the control-plane's validation
config
+    isn't forwarded ? can REJECT a clip the gate already passed (observed 2026-07-07: passed at
+    validation.min_blur_threshold=8.0, then worker-rejected at its default 20.0). So the
dispatch MUST
+    carry --skip-validation, telling the worker to trust the gate. Regression guard for that

```

```

wiring."""
+   _db, _bucket, monkeypatch = cloud_env
+   fake = _FakeCloudClient()
+   _inject_fake(monkeypatch, fake)
+   m._execute_processing(
+       m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
+   )
+   assert "--skip-validation" in fake.calls[0]
+
+
+   def test_cloud_failure_marks_failed_no_rows(cloud_env):
+       db, _bucket, monkeypatch = cloud_env
+       # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-confirm).
diff --git a/tests/test_worker.py b/tests/test_worker.py
index 8fb5e16..72b2514 100644
--- a/tests/test_worker.py
+++ b/tests/test_worker.py
@@ -97,6 +97,14 @@ def test_parser_infer_accepts_set_and_uris():
     assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]

+def test_parser_infer_skip_validation_flag():
+    base = ["infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b"]
+    # Default-OFF: the direct `worker infer` CLI still validates.
+    assert wmain.build_parser().parse_args(base).skip_validation is False
+    # The dispatcher passes it to disable the redundant worker re-validation.
+    assert wmain.build_parser().parse_args(base + ["--skip-validation"]).skip_validation is
True
+
+
+    # ----- cmd_infer (local, mocked)

@@ -407,6 +415,43 @@ def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
    assert marker["returncode"] == 2 and marker["status"] == "failed"

+def test_cmd_infer_skip_validation_bypasses_validators(tmp_path, monkeypatch):
+    """--skip-validation (set by the dispatcher, the authoritative gate) must NOT run the
worker's
+    validators ? no second decode, and no chance of a threshold mismatch REJECTING a clip the
+    control-plane already passed (the live 2026-07-07 CUJ failure). Prove it by making
+    run_all_with_context RAISE: a skipped run never touches it, so the job completes rc 0 with
a
+    status="success" report. Contrast test_cmd_infer_validation_failure_returns_2 (no flag ? rc
2)."""
+    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidSkip")
+
+    def _boom(*a, **k): # the validators must never be reached on the skip path
+        raise AssertionError("run_all_with_context must not run under --skip-validation")
+
+    monkeypatch.setattr(wmain.validator_registry, "run_all_with_context", _boom)
+    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
+    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
+    video = tmp_path / "in.mp4"
+    video.write_bytes(b"FAKE")
+    out = tmp_path / "bucket"
+
+    rc = wmain.main(
+        [
+            "infer",
+            "--video-uri",
+            str(video),
+            "--out-uri",
+            str(out),

```

```

+         "--scratch",
+         str(tmp_path / "s"),
+         "--segmenter",
+         "wasb_hybrid",
+         "--skip-validation",
+     ]
+ )
+ assert rc == 0 # a clip that WOULD fail re-validation still succeeds ? the gate is trusted
+ report = json.loads((out / "outputs" / "vidSkip" / "report.json").read_text())
+ assert report["status"] == "success" and report["segment_count"] == 2
+
+ # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle ---

```

4. assistant (2026-07-07T14:50:43.266Z)

Let me examine the key files referenced in the finding.

5. user (2026-07-07T14:50:44.234Z)

```

1     """`CloudRunCompute` ? dispatch one inference job to a Cloud Run GPU Job and
bucket-poll
2     for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4     Flow (the cloud-serving compute path):
5     1. Dispatch a Cloud Run Job execution whose container runs `python -m
backend.worker infer
6         --video-uri <gs> --out-uri <gs> --done-uri <gs> ?` (the SAME worker, on the L4
GPU).
7     2. Bucket-poll `done_uri` via `execution_policy.poll_until` + the storage
layer (D16 ? the
8         completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9         it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10        cold-start is absorbed by the async poll. On a poll TIMEOUT (`max_wait_sec` can
undercut the
11        job's own `--task-timeout` budget) the shim does ONE final marker check ? a slow-
but-healthy
12        worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13        the still-running execution so the L4 doesn't keep billing until `--task-
timeout`.
14    3. Read the marker (it carries `video_id` + the worker's returncode) ? fetch
15        `outputs/<video_id>/report.json` ? `ComputeResult`.
16
17    The dispatched container is forced to run INLINE (`compute_target=local_gpu` +
native detector)
18    so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
19
20    The Cloud Run trigger is an injected client (`CloudRunJobClient`) so the whole
shim is unit-tested
21    with a fake (no GCP). The real `GoogleCloudRunJobClient` lazy-imports `google-cloud-
run` and is
22    owner-verified on the M7 real run.
23    """
24
25    from __future__ import annotations
26
27    import json
28    import logging
29    import os
30    import tempfile
31    import uuid

```

```

32     from abc import ABC, abstractmethod
33     from typing import Any, Callable, List, Optional
34
35     from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
36     from backend.infrastructure.execution_policy import (
37         DEFAULT_POLICY,
38         ExecutionPolicy,
39         PolicyTimeout,
40         poll_until,
41     )
42     from backend.storage import fetch, get_storage, parse_uri
43
44     LOG = logging.getLogger("compute.cloud_run")
45
46     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
47     # branch.
48     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
49     # runner.)
50     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
51         "deployment.compute_target=local_gpu",
52         "indexing.detector_impl=native",
53     )
54
55     class CloudRunJobClient(ABC):
56         """Triggers a Cloud Run **Job** execution with per-execution container arg
57         overrides.
58
59         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
60         implementation
61         overrides the job's container ``args`` for this execution and starts it."""
62
63         @abstractmethod
64         def run_job(
65             self,
66             job_name: str,
67             argv: List[str],
68             *,
69             region: str,
70             project: Optional[str] = None,
71         ) -> str:
72             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
73             return an
74             execution id/name (for logs). Does NOT block on completion (the caller bucket-
75             polls)."""
76
77         @abstractmethod
78         def cancel_execution(self, execution: str) -> None:
79             """Cancel a running execution by the name ``run_job`` returned. Called when the
80             bucket-poll
81             times out so the abandoned execution doesn't keep billing the GPU until
82             ``--task-timeout``.
83             May raise ? the caller treats every failure as best-effort (the original poll
84             timeout is
85             NEVER masked by a cancel error)."""
86
87     class GoogleCloudRunJobClient(CloudRunJobClient):
88         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
89         run; CI uses
90         a fake, so this code path is never exercised in tests (the SDK isn't a test
91         dependency)."""
92
93         def run_job(
94             self,

```

```

86         job_name: str,
87         argv: List[str],
88         *,
89         region: str,
90         project: Optional[str] = None,
91     ) -> str:
92         # A None/empty project would build a bogus "projects/None/locations/..." path
93         (job_path
94         # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
95         exercisable
96         # in CI (where google-cloud-run is absent) and so mypy narrows project to str
97         for job_path.
98         if not project:
99             raise RuntimeError(
100                 "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
101                 project id "
102                 "to resolve the job path; set it on the control plane (see
103                 get_compute_backend)."
104             )
105         try:
106             from google.cloud import run_v2 # type: ignore[attr-defined]
107         except Exception as exc: # pragma: no cover - real SDK not present in CI
108             raise RuntimeError(
109                 "google-cloud-run is required for CloudRunCompute's real client; install
110                 it on the "
111                 "control plane (it is intentionally NOT a CI/test dependency)."
112             ) from exc
113         client = (
114             run_v2.JobsClient()
115         ) # pragma: no cover - exercised only on the real M7 run
116         name = client.job_path(project, region, job_name)
117         overrides = run_v2.RunJobRequest.Overrides(
118             container_overrides=[
119                 run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
120             ]
121         )
122         op = client.run_job(
123             request=run_v2.RunJobRequest(name=name, overrides=overrides)
124         )
125         return (
126             getattr(op, "metadata", None)
127             and getattr(op.metadata, "name", "")
128             or "execution"
129         )
130
131     def cancel_execution(self, execution: str) -> None:
132         # run_job falls back to the literal string "execution" when the operation
133         metadata carries
134         # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
135         import
136         # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
137         (where
138         # google-cloud-run is absent).
139         if "/executions/" not in (execution or ""):
140             raise RuntimeError(
141                 f"not a fully-qualified Cloud Run execution name: {execution!r} ?
142                 expected "
143                 "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
144                 resolve "
145                 "one from the operation metadata, so there is nothing to cancel by
146                 name)."
147             )
148         try:
149             from google.cloud import run_v2 # type: ignore[attr-defined]
150         except Exception as exc: # pragma: no cover - real SDK not present in CI

```



```

139         raise RuntimeError(
140             "google-cloud-run is required to cancel a Cloud Run execution; install
it on the "
141             "control plane (it is intentionally NOT a CI/test dependency)."
142         ) from exc
143     client = (
144         run_v2.ExecutionsClient()
145     ) # pragma: no cover - exercised only on a real run
146     client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
147
148
149 class CloudRunCompute(ComputeBackend):
150     """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
151
152     def __init__(
153         self,
154         *,
155         run_client: CloudRunJobClient,
156         job_name: str,
157         region: str,
158         project: Optional[str] = None,
159         storage_config: Optional[dict] = None,
160         policy: ExecutionPolicy = DEFAULT_POLICY,
161         token: Optional[str] = None,
162         container_overrides: Optional[tuple[str, ...]] = None,
163     ) -> None:
164         self._client = run_client
165         self._job_name = job_name
166         self._region = region
167         self._project = project
168         self._storage_config = storage_config or {}
169         self._policy = policy
170         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
171         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
172         self._token = token
173         self._container_overrides = (
174             _DEFAULT_CONTAINER_OVERRIDES
175             if container_overrides is None
176             else tuple(container_overrides)
177         )
178
179     def run_infer(
180         self,
181         spec: ComputeJobSpec,
182         *,
183         on_wait: "Callable[[float], None] | None" = None,
184     ) -> ComputeResult:
185         out_root = spec.out_uri.rstrip("/")
186         token = self._token or uuid.uuid4().hex
187         done_uri = f"{out_root}/_dispatch/{token}.done"
188         argv: List[str] = [
189             "infer",
190             "--video-uri",
191             spec.video_uri,
192             "--out-uri",
193             spec.out_uri,
194             "--done-uri",
195             done_uri,
196         ]
197         if spec.segmenter:
198             argv += ["--segmenter", spec.segmenter]
199         if spec.skip_validation:
200             # The control-plane already validated + only dispatches on PASS ? tell the

```

```

worker not to
201         # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
202         argv.append("--skip-validation")
203     for kv in (*spec.config_overrides, *self._container_overrides):
204         argv += ["--set", kv]
205
206     execution = self._client.run_job(
207         self._job_name, argv, region=self._region, project=self._project
208     )
209     LOG.info(
210         "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
211         self._job_name,
212         execution,
213         done_uri,
214     )
215
216     try:
217         marker = poll_until(
218             lambda: self._read_marker(done_uri),
219             self._policy,
220             label="cloud-run-infer",
221             logger=LOG,
222             # Live heartbeat (#482): each still-waiting tick reaches the caller so
223             # job.progress moves during the whole GPU run instead of freezing.
224             on_tick=on_wait,
225         )
226     except PolicyTimeout as timeout_exc:
227         marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
228         video_id = str(marker.get("video_id", ""))
229         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
230         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
231         rc = int(marker.get("returncode", 1))
232         if not video_id:
233             LOG.warning(
234                 "cloud-run: job=%s completion marker present but empty/unreadable (no "
235                 "video_id) ? treating as failure",
236                 self._job_name,
237             )
238             rc = rc or 1
239         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
240         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
241         LOG.info(
242             "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
243         )
244         return ComputeResult(
245             returncode=rc,
246             outputs_uri=outputs_uri,
247             video_id=video_id,
248             report=report,
249             execution=str(execution or ""),
250         )
251
252     def _handle_poll_timeout(
253         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
254     ) -> dict:
255         """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
256
257         The poll budget (`max_wait_sec`, default 1800s) can undercut the deployed
job's own
258         ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
259         ONE final marker check rescues that completed GPU run instead of wasting it. If

```

```

the marker
260         is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
261         L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
262         execution name so the operator can verify the state in the GCP console. A cancel
failure
263         NEVER masks the original timeout."""
264         try:
265             late = self._read_marker(done_uri)
266         except Exception: # noqa: BLE001 ? the final check is best-effort too: absence
? failure
267             LOG.warning(
268                 "cloud-run: final post-timeout marker check failed ? treating as
absent",
269                 exc_info=True,
270             )
271             late = None
272         if late is not None:
273             LOG.warning(
274                 "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
275                 "? rescuing the completed result",
276                 self._job_name,
277                 execution,
278             )
279             return late
280         try:
281             self._client.cancel_execution(execution)
282         except Exception as cancel_exc: # noqa: BLE001 ? best-effort: never mask the
timeout
283             cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
284             LOG.warning(
285                 "cloud-run: best-effort cancel of execution=%s failed: %s",
286                 execution,
287                 cancel_exc,
288                 exc_info=True,
289             )
290         else:
291             cancel_note = "cancel requested"
292             LOG.info("cloud-run: requested cancel of execution=%s", execution)
293         raise PolicyTimeout(
294             f"{timeout_exc} [cloud-run job={self._job_name} "
295             f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
296             "state in the GCP console]"
297         ) from timeout_exc
298
299         # --- storage-backed completion signal (bucket-poll) ---
300         def _read_marker(self, done_uri: str) -> Optional[dict]:
301             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
302             ref = parse_uri(done_uri)
303             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
304                 return None
305             return (
306                 self._read_json(done_uri) or {}
307             ) # empty dict still terminates the poll (present == done)
308
309         def _read_json(self, uri: str) -> dict:
310             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
311             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
312             # would silently return {} for every gs://|s3:// object (the only backends this

```

```

shim targets).
313         try:
314             ref = parse_uri(uri)
315             with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
316                 dst = os.path.join(td, ref.name or "obj.json")
317                 local = fetch(uri, dst, config=self._storage_config)
318                 with open(local, "r", encoding="utf-8") as f:
319                     data: Any = json.load(f)
320                 return data if isinstance(data, dict) else {}
321         except Exception:
322             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
323         return {}
324

```

6. user (2026-07-07T14:50:52.060Z)

```

760         help="prefix containing labels/ (+ videos/ for "
761         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
762     )
763     pt.add_argument(
764         "--out-uri",
765         required=True,
766         help="output prefix (weights/<version>/ is appended)",
767     )
768     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
769     pt.add_argument(
770         "--trajectory-source",
771         choices=["synth", "detector"],
772         default="synth",
773         help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
774         "detector = REAL shuttle trajectories via the resolved DetectorRunner "
775         "(needs videos/ in the corpus; honours detector_impl).",
776     )
777     pt.add_argument(
778         "--segmenter",
779         default="wasb_hybrid",
780         help="trajectory hybrid whose detector is used for --trajectory-source
detector",
781     )
782     pt.add_argument(
783         "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
784     )
785     pt.add_argument("--config", default=None)
786     pt.add_argument("--scratch", default=None)
787     pt.add_argument(
788         "--fps",
789         type=float,
790         default=30.0,
791         help="synth trajectory fps (detector probes the video)",
792     )
793     pt.add_argument(
794         "--frame-width",
795         type=float,
796         default=1280.0,
797         help="synth trajectory frame width (detector probes the video)",
798     )
799     pt.add_argument("--set", action="append", default=[], metavar="k=v")
800     pt.set_defaults(func=cmd_train)
801     return p
802
803
804     def _setup_logging(verbose: bool) -> None:
805         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
806         VISIBLE ? the worker previously ran with no logging config, so every

```

```

segmenter/detector log
807         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
808         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
809         entry), not at import, so importing the module never reconfigures a caller's
logging.""
810         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
811         # configured logging at import), which would leave the level at WARNING and re-
suppress the
812         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
813         logging.basicConfig(
814             level=logging.DEBUG if verbose else logging.INFO,
815             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
816             datefmt="%H:%M:%S",
817             force=True,
818         )
819
820
821     def main(argv=None) -> int:
822         args = build_parser().parse_args(argv)
823         _setup_logging(getattr(args, "verbose", False))
824         return args.func(args)
825
826
827     if __name__ == "__main__":
828         sys.exit(main())
829

```

7. user (2026-07-07T14:51:00.228Z)

```

240         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
241         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
242         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
243         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
244         compute = get_compute_backend(config)
245         if compute is not None:
246             return _dispatch_remote(compute, args)
247
248         storage_cfg = config.storage.model_dump()
249
250         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
251         os.makedirs(scratch, exist_ok=True)
252         config.output.output_dir = scratch
253
254         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
255         local_video = storage.fetch(
256             args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
257         )
258         print(f"[worker] pulled {args.video_uri} -> {local_video}")
259
260         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
261         video_id = compute_video_id(local_video)
262
263         # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
264         # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the

```

```

265         # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
266         # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
267         # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
268         # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
269         # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
270         # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
271         # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
272         if getattr(args, "skip_validation", False):
273             print(
274                 "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
275                 "gate and already validated this input."
276             )
277             val_results, val_context = [], {}
278         else:
279             val_results, val_context = validator_registry.run_all_with_context(
280                 local_video, config
281             )
282         failures = [r for r in val_results if not r.passed]
283         db = Database(os.path.join(scratch, config.output.db_name))
284         db.add_video(
285             video_id,
286             local_video,
287             "failed" if failures else "processed",
288             [r.model_dump() for r in val_results],
289         )
290
291         def _fail(rc: int) -> int:
292             # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
293             # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
294             # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
295             # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
296             # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
297             # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
298             # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
299             try:
300                 _serialize_and_push_outputs(
301                     scratch,
302                     args.out_uri,
303                     video_id,
304                     [],
305                     "failed",
306                     storage_cfg,
307                     str(getattr(args, "video_uri", "")) or "",
308                 )
309             except Exception as e: # never mask the real failure on an artifact-push hiccup
310                 logger.warning("[worker] could not push failure artifacts: %s", e)
311             # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
312             if getattr(args, "done_uri", None):
313                 _write_done_marker(
314                     args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch

```

```

315         )
316         return rc
317
318     segments: List[dict] = []
319     segmenter_actual = None
320     segmentation_ran = False
321     status = "failed"
322     try:
323         if failures:
324             print(
325                 "[worker] validation FAILED: "
326                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
327             )
328             return _fail(2)
329         seg_name = args.segmenter or config.indexing.default_segmenter
330         segmenter_cls = segmenter_registry.get(seg_name)
331         if not segmenter_cls:
332             print(
333                 f"[worker] unknown segmenter: {seg_name!r} "
334                 f"(have {list(segmenter_registry.list_available())})"
335             )
336             return _fail(2)
337         segmenter_actual = seg_name
338         result = segmenter_cls(db, config).process_video(
339             video_id, local_video, max_frames=args.max_frames
340         )
341         segmentation_ran = True
342         if isinstance(result, Failure):
343             print(f"[worker] segmenter FAILED: {result.message}")
344             return _fail(3)
345         segments = result.value if hasattr(result, "value") else result
346         status = "success"
347         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
348         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
349         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
350         if not segments:
351             detector_impl = (
352                 os.environ.get("RALLY_DETECTOR_IMPL")
353                 or getattr(config.indexing, "detector_impl", None)
354                 or "auto"
355             )
356             logger.warning(
357                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
358                 "The detector substrate yielded no usable trajectories/windows ? check
the detector "
359                 "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "

```

8. assistant (2026-07-07T14:51:05.192Z)

The done-marker is written only inside `cmd\_infer`. Now let me verify the ENTRYPOINT and the deploy README, and check the git history claims about #507/#509.

9. user (2026-07-07T14:51:13.425Z)

```

=== find Dockerfiles / entrypoints ===
deploy/cloudrun/Dockerfile
deploy/cloudrun/README.md
deploy/cloudrun/cloudbuild.yaml
deploy/cloudrun/gen_service_yaml.py
deploy/cloudrun/post_deploy_smoke.ps1
deploy/digitalocean/README.md

```

```

deploy/digitalocean/bootstrap.sh
deploy/digitalocean/gpu_setup.sh
deploy/digitalocean/mount_scratch.sh
deploy/gcp/README.md
deploy/gcp/buckets.env
deploy/gcp/create_gpu_vm.sh
deploy/gcp/nightly_regression/Dockerfile.launcher
deploy/gcp/nightly_regression/README.md
deploy/gcp/nightly_regression/cloudbuild.launcher.yaml
deploy/gcp/nightly_regression/create_nightly_bucket.sh
deploy/gcp/nightly_regression/launch.sh
deploy/gcp/nightly_regression/register_scheduler.sh
deploy/gcp/nightly_regression/run_on_vm.sh
deploy/gcp/nightly_regression/setup_email.sh
deploy/gcp/ops-agent-startup.sh
deploy/gcp/provision.sh
deploy/gcp/teardown.sh
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_2026-07-04.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_alpha-all1v1_2026-07-06.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_cp-rebuild_2026-07-05.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_gpu-resident-
nvdec_2026-07-07.md
docs/archives/past_projects/DECOUPLED_COMPUTE/DEPLOY_REPORT_GCP_2026-06-20.md
tests/test_config_deployment.py
tests/test_deploy_manifests.py
tests/test_server_entrypoint.py
=== README section 2a ===
cloudbuild.yaml
Dockerfile
gen_service_yaml.py
post_deploy_smoke.ps1
README.md

```

10. user (2026-07-07T14:51:17.706Z)

```

1      # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2      #
3      # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4      # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5      # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6      # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7      #
8      # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9      # native GPU box, deploy/digitalocean/gpu_setup.sh):
10     #   * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11     #   * a conda `wasb` env (py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec ? the #506
modernized stack,
12     #       validated on a real L4: cpu-decode F1 0.576 ? the torch-1.11 baseline 0.582) runs
the WASB
13     #       shuttle detector as a subprocess (the native runner shells out to $WASB_PYTHON),
so the app's
14     #       torch never co-installs with it.
15     #
16     # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
17     #       commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
18     #       mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner

```



```

serving.
19
20 # Driver userspace libs for NVDEC/NVENC (#506): GPU hosts inject the COMPUTE driver
stack but not
21 # the video-codec libs ? no libnvcuvid.so.1, no libnvidia-encode.so.1 ? and
PyNvVideoCodec needs
22 # BOTH (it initializes NVENC even for decode-only). Extract the pair from the matching
datacenter
23 # driver .run in a throwaway stage. Keep NVIDIA_DRIVER_VERSION aligned with the GPU
fleet's driver
24 # (580.159.03 = the validated L4 combo); these userspace libs talk to the host-injected
libcuda,
25 # so a large version skew can break decode ? the golden gate on the real image is the
check.
26 ARG NVIDIA_DRIVER_VERSION=580.159.03
27 FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04 AS nvlibs
28 ARG NVIDIA_DRIVER_VERSION
29 RUN apt-get update && apt-get install -y --no-install-recommends curl ca-certificates \
30     && rm -rf /var/lib/apt/lists/* \
31     && curl -fsSL -o /tmp/drv.run \
32     "https://us.download.nvidia.com/tesla/${NVIDIA_DRIVER_VERSION}/NVIDIA-
Linux-x86_64-${NVIDIA_DRIVER_VERSION}.run" \
33     && sh /tmp/drv.run --extract-only --target /tmp/drv \
34     && mkdir -p /nvlibs \
35     && cp -a /tmp/drv/libnvcuvid.so.* /tmp/drv/libnvidia-encode.so.* /nvlibs/ \
36     && rm -rf /tmp/drv /tmp/drv.run
37
38 FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
39
40 ENV DEBIAN_FRONTEND=noninteractive \
41     PYTHONUNBUFFERED=1 \
42     PIP_NO_CACHE_DIR=1 \
43     CONDA_DIR=/opt/conda \
44     WASB_REPO_DIR=/opt/models/WASB-SBDT
45
46 # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
47 RUN apt-get update && apt-get install -y --no-install-recommends \
48     ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
49     python3 python3-pip python3-venv \
50     && rm -rf /var/lib/apt/lists/*
51
52 # NVDEC/NVENC userspace libs (#506, from the nvlibs stage above) + the .so.1 sonames
53 # PyNvVideoCodec dlopens. Harmless when decode_backend=cpu; required for nvdec_resident.
54 ARG NVIDIA_DRIVER_VERSION
55 COPY --from=nvlibs /nvlibs/ /usr/lib/x86_64-linux-gnu/
56 RUN ln -sf "libnvcuvid.so.${NVIDIA_DRIVER_VERSION}" /usr/lib/x86_64-linux-
gnu/libnvcuvid.so.1 \
57     && ln -sf "libnvidia-encode.so.${NVIDIA_DRIVER_VERSION}" /usr/lib/x86_64-linux-
gnu/libnvidia-encode.so.1 \
58     && ldconfig
59
60 # --- WASB detector env (micromamba + conda-forge ? BSD-licensed, NO Anaconda ToS; P3a,
ADR-005) ---
61 # Enforces ADR-005 (every dependency ? code, data, weights, AND the build toolchain ?
stays
62 # MIT/Apache/BSD): micromamba (BSD-3) + conda-forge (community) replaces Miniconda's
Anaconda-ToS-gated
63 # `defaults` channels (the ?200-employee Terms-of-Service trap flagged in
PACKAGING_PLAN.md §3).
64 # MAMBA_ROOT_PREFIX=$CONDA_DIR (= /opt/conda) keeps the env at $CONDA_DIR/envs/wasb so
$WASB_PYTHON below
65 # is UNCHANGED.
66 #
67 # #506 modernized stack (validated end-to-end on a real L4, 2026-07-06): py3.10 + torch

```

```

2.6.0+cu124
68     # (torch ?2 needs py?3.9; WASB HRNet ports clean ? cpu F1 0.576 ? the torch-1.11
baseline 0.582) +
69     # PyNvVideoCodec (MIT ? the practical NVDEC path; TorchCodec needs a matching ffmpeg
build and could
70     # not load) for decode_backend=nvdec_resident. wasb_infer.py carries the two version
shims this bump
71     # needs (torch.load weights_only=False; the NumPy-2.0 np.Inf alias for WASB's tracker).
numpy is
72     # UNPINNED (the old "numpy<1.24" was the torch-1.11 ABI pin). WASB-SBDT ships no
requirements.txt ?
73     # the explicit list below is the (L4-validated) dependency closure its src/ imports
need.
74     ENV MAMBA_ROOT_PREFIX=/opt/conda
75     RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest \
76         | tar -xj -C /usr/local bin/micromamba \
77         && micromamba create -y -q -n wasb -c conda-forge python=3.10 \
78         && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
79         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
80             torch==2.6.0 torchvision==0.21.0 --index-url
https://download.pytorch.org/whl/cu124 \
81         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q PyNvVideoCodec==2.1.0 \
82         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
83             hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib numpy
84
85     # --- Main application env ---
86     WORKDIR /app
87     COPY pyproject.toml README.md ./
88     COPY requirements-trackers.txt ./
89     COPY backend ./backend
90     COPY training ./training
91     # Install with the [storage-gcs,cloud-run,auth] extras. storage-gcs: the cloud worker
PULLS the gs://
92     # input and PUSHES the gs:// outputs/done-marker through backend.storage (google-cloud-
storage is
93     # lazy-imported there); without it the job dies at the first gs:// fetch with
`ModuleNotFoundError:
94     # google.cloud`. cloud-run: this SAME image is reused as the deployed CONTROL PLANE,
which dispatches
95     # the L4 worker Job via google.cloud.run_v2 (backend/compute/cloud_run.py, lazy-
imported); without it
96     # the control plane fails every dispatch with "google-cloud-run is required for
CloudRunCompute's real
97     # client". Both are real Cloud Run failures the mock-client tests can't see (the SDKs
are faked in CI).
98     # auth: PyJWT[crypto] for M9 edge-auth (security.edge_auth_enabled ? the Cf-Access JWT
verify the
99     # reused-as-control-plane deployment runs behind api.khelsutra.guru); jwt is lazy-
imported and
100    # edge-auth is default-OFF, so the worker path is unaffected ? but without it a control-
plane
101    # deploy with edge-auth ON needed a hand-built derived image (the A1 deploy's PyJWT
overlay).
102    # D13/P12: `trackers` (ByteTrack's new home) is a git-only dep (broken PyPI wheel ?
103    # roboflow/trackers#474) so it's NOT pulled by `pip install .`; install it out-of-band
before
104    # the main env, or the person-cue path crashes with ModuleNotFoundError. The immutable
git SHA is
105    # single-sourced in requirements-trackers.txt.
106    RUN python3 -m pip install --upgrade pip \
107        && python3 -m pip install -r requirements-trackers.txt \
108        && python3 -m pip install ".[storage-gcs,cloud-run,auth]"
109
110    # The native runner finds the WASB env + repo by env (no `conda activate` needed);

```

```

weights come in at
111 # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
112 # PYTHONUNBUFFERED: without it the worker's stdout/stderr sit in the block buffer until
process
113 # exit, so Cloud Logging shows a single end-of-run burst instead of live progress
(observed on the
114 # first real M7 runs, 2026-07-03) ? a running job looks stalled and mid-run failures log
nothing.
115 ENV PYTHONUNBUFFERED=1 \
116     WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
117     WASB_REPO_DIR=/opt/models/WASB-SBDT \
118     WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
119     RALLY_DETECTOR_IMPL=native
120
121 # The dispatcher (CloudRunCompute) supplies the per-execution args:
122 # infer --video-uri gs:/// --out-uri gs:/// --done-uri gs:/// --segmenter wasb_hybrid
--set ?
123 ENTRYPOINT ["python3", "-m", "backend.worker"]
124

```

11. assistant (2026-07-07T14:51:26.692Z)

ENTRYPOINT is bare `python3 -m backend.worker` ? confirmed, no wrapper. Now the README §2a and how #507 handled the same skew.

12. user (2026-07-07T14:51:27.687Z)

```

1 # Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)
2
3 The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
4 dispatches a Cloud Run Job execution running this image's worker on an L4, then
bucket-polls
5 a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.
6
7 > Status: the dispatch shim + image are landed + mock-tested (M6). The real
build + push +
8 > run on an L4 is the M7 owner step, within the $100/?10k budget (D17) ?
confirm cmd + hourly
9 > cost first, delete every resource after ([[gcp-vm-always-delete]]). CI does
not build this image.
10
11 ## 0. Prerequisites (owner)
12 - A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
13 - GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else
Singapore ? see
14 `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15 - The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16
17 ## 1. Build + push the image (Artifact Registry)
18 ``bash
19 PROJECT=<gcp-project>; REPO=rally
20 # L4 GPU is region-limited ? use asia-southeast1 (Singapore), NOT asia-south1 (Mumbai).
21 REGION=asia-southeast1
22 gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
23 # `gcloud builds submit` has NO -f flag, so a non-root Dockerfile needs a build config:
24 gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project $PROJECT .
25 ``
26 *(A CUDA + micromamba image is large; the first build is ~8?10 min. Budget-watch per
D17.)*
27

```

```

28     ## 2. Create the Cloud Run **Job** (GPU, scale-to-zero)
29     The proven shape (first real M7 deploy, 2026-07-03): weights staged in a **same-region
bucket**
30     mounted read-only via GCS FUSE (no weights in the image ? WASB-C3; no init-fetch step
needed).
31     ```bash
32     gcloud run jobs create rally-worker \
33         --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
34         --region $REGION --gpu 1 --gpu-type nvidia-l4 --no-gpu-zonal-redundancy \
35         --cpu 8 --memory 32Gi \
36         --max-retries 0 --task-timeout 3600 \
37         --set-env-vars WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar \
38         --add-volume name=serving,type=cloud-storage,bucket=$SERVING_BUCKET \
39         --add-volume-mount volume=serving,mount-path=/gcs
40     ```
41     - **`--no-gpu-zonal-redundancy` is REQUIRED on the CLI**, not just a cost choice (it
also halves
42     the GPU rate ? right for batch jobs): without the flag `gcloud` asks an interactive
Y/n which,
43     in a non-interactive shell, **hangs forever with zero output and nothing created**. An
empty
44     `gcloud` that "did nothing" ? suspect a hidden prompt.
45     - 8 vCPU / 32 GiB: x264 encode is CPU-side, so CPU headroom keeps the paid GPU busy.
`/tmp` is
46     **RAM-backed** ? the pulled input video lives in memory, so size `--memory` for the
largest
47     expected upload (a 5.8 GB GoPro file ? 6 GiB of tmpfs).
48
49     ## 2a. GPU-resident WASB decode (#506/#507) ? the current serving default
50     The image ships the **py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec==2.1.0** WASB env with
51     `libnvcuvid`/`libnvidia-encode` **baked in** (multi-stage layer from the matching NVIDIA
driver ?
52     Cloud Run injects the host driver, not these userspace codec libs).
`indexing.wasb.decode_backend`
53     selects the decode path:
54     - `cpu` (model default) ? the historical cv2 decode; **byte-unchanged** by this work.
55     - `nvdec_resident` ? NVDEC decode + cv2-faithful affine `grid_sample` resize + GPU
normalize, all
56     on-GPU. **The cloud-serving preset default** (validated: offline golden gate @ SERVED
F1 0.574 ?
57     baseline 0.582, AND on the live L4 the served GX010128 trajectory reproduced the VM
run ?
58     26 479 ? 26 481 visible). CUDA-only (config-validator-enforced).
59
60     **How the knob reaches the worker.** The worker Job runs with NO `DEPLOYMENT_PROFILE`,
so it does
61     **not** apply the cloud-serving preset itself. The control plane (which *does* have the
profile)
62     resolves the config and **forwards**
`indexing.wasb.{decode_backend,batch_size,prefetch_batches}`
63     to the worker as `--set` overrides (`orchestration.py`, alongside `skip_ai_handoff`). So
the
64     **preset is the single source of truth** ? changing `presets.py` + redeploying the
control plane
65     changes serving; no per-Job knob needed.
66
67     **Override / rollback lever (no redeploy).** The native runner reads
`WASB_DECODE_BACKEND` FIRST,
68     so a Job env var wins over the forwarded `--set`:
69     ```bash
70     # force nvdec (or pin cpu) on the Job regardless of the forwarded config; instant
rollback = remove it
71     gcloud run jobs update rally-worker --region $REGION --update-env-vars
WASB_DECODE_BACKEND=nvdec_resident
72     gcloud run jobs update rally-worker --region $REGION --remove-env-vars

```

```

WASB_DECODE_BACKEND # ? forwarded/preset default
73 # full revert: pin the prior image by digest
74 gcloud run jobs update rally-worker --region $REGION --image $REGION-
docker.pkg.dev/$PROJECT/$REPO/rally-worker@sha256:<prev>
75 ```
76 ? Forwarding `decode_backend=nvdec_resident` requires a worker image that understands
77 `--decode-backend` (? #507). Roll the **worker image and the control plane together** ?
a new
78 control plane forwarding nvdec to an old worker image argparse-fails (rc 2).
79
80 **Post-deploy smoke.** After a cutover, confirm the real L4 path with
81 `deploy/cloudrun/post_deploy_smoke.ps1` (run from PowerShell ? see $Gotchas on MSYS arg-
mangling):
82 ```powershell
83 ./deploy/cloudrun/post_deploy_smoke.ps1 -Video gs://khelsutra-rally-
corpus/videos/mahadevpura_smoke_180s.mp4 -Decode cpu # image sanity (fast)
84 ./deploy/cloudrun/post_deploy_smoke.ps1 -Video gs://khelsutra-rally-
corpus/videos/GX010128.MP4 -Decode env # nvdec via preset/env (hard clip)
85 ```
86
87 ## 3. Wire the dispatcher (control plane)
88 The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
89 `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
90 ```bash
91 export DEPLOYMENT_PROFILE=cloud-serving
92 export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
93 # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
94 # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
95 python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
96 ```
97 **The AI handoff needs its key IN THE JOB env** (`GEMINI_API_KEY` ? mount a Secret
Manager secret
98 on the job), or run fully-local with `--set indexing.skip_ai_handoff=true` (candidate
windows
99 emitted as rallies ? the eval-parity mode). With neither, the segmenter used to bail to
a silent
100 0-rally "success" **after** the paid GPU pass; the cloud-serving startup checks now fail
this at
101 boot (`AI_HANDOFF_KEY_MISSING`, rc 2) before the video is even pulled.
102
103 ### 3a. Control-plane SERVICE deploy ? required flags for reliable async jobs
104
105 The API server (`/api/process`, `/api/compile` ? #329) runs its **job worker IN-
PROCESS**: the
106 request returns `202` and a background thread drains the job (GPU detection dispatches
to the Job;
107 the reel stitch runs on the control plane itself). On Cloud Run that background thread
only makes
108 progress while the instance has CPU, so a scale-to-zero **service** deploy MUST set:
109
110 **The canonical deploy path is the committed manifest generator** (#473) ? it bakes the
111 cloud-serving config.json into a startup wrapper, sets every flag below, pins
`maxScale=1`
112 (per-instance SQLite state, issue #471), and deliberately does NOT set `CLOUD_RUN_JOB`
(a
113 reserved runtime env name that gets a manifest rejected; the code default is already
right):
114 ```bash
115 python deploy/cloudrun/gen_service_yaml.py --edge-auth on -o service.yaml

```

```

116     gcloud run services replace service.yaml --region $REGION # from PowerShell on Windows
117     ```
118     `--edge-auth off` is only for the pre-CF identity-token smoke/e2e window; redeploy with
119     `on` before testers touch the service again. The flag-based equivalent (for reference ? a
manifest
120     round-trips the bash wrapper's metacharacters, flags don't on Windows):
121     ```bash
122     gcloud run deploy rally-control-plane --image <image> --region $REGION \
123         --no-cpu-throttling \          # CPU stays allocated between requests ? the drain thread
finishes
124         --min-instances 1 \          # one always-warm instance drains reliably (cheap CPU;
NOT the D7 GPU pool)
125         --cpu 2 --memory 4Gi --port 8080 --allow-unauthenticated \
126         --set-env-vars DEPLOYMENT_PROFILE=cloud-
serving,CLOUD_RUN_REGION=$REGION,GOOGLE_CLOUD_PROJECT=$PROJECT
127     ```
128     Without `--no-cpu-throttling`, a `/api/compile` returns 202 and then **stalls** ? the
reel never
129     finishes and startup crash-recovery would mark it terminal. `min-instances 1` does NOT
contradict
130     D7 (that was the ~$500/mo warm **GPU** pool); this is a ~1-vCPU CPU instance. Defense-
in-depth is in
131     the code: `Database.recover_stale_jobs()` **re-queues** a compile interrupted by a
restart (idempotent
132     stitch) rather than failing it, so even an unlucky scale-down mid-stitch self-heals on
the next drain.
133     Reserve `--allow-unauthenticated` for the pre-CF-auth smoke test only; lock ingress to
the CF proxy
134     (M9) before inviting testers. The image ships the `auth` extra (PyJWT[crypto]), so
flipping
135     `security.edge_auth_enabled` on needs no derived image ? the M9 Cf-Access JWT verify
works out of
136     the box.
137
138     ## 4. M7 acceptance (the owner runlog)
139     Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
140     `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
commands,
141     rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
(`ffmpeg -f null -`
142     exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
143
144     ## Gotchas
145     - The dispatched container **must run inline** ? `CloudRunCompute` forces
`compute_target=local_gpu`
146     + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
147     - Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
148     ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
control-plane restart.
149     - A **failed** Job (validation rc 2 / segmenter rc 3) now writes a **failure marker**
carrying the
150     real non-zero rc, so the dispatcher's poll terminates **FAST + accurately** (no 30-min
hang on a bad
151     video). Only a **hung/crashed** container (no marker at all) makes the bounded poll
time out
152     (`ExecutionPolicy.max_wait_sec`); the worker CLI maps that to a clean **rc 4**, and
the API path
153     surfaces it as a `cloud dispatch error` job failure (not a traceback).
154     - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
libx264.
155     - **`GOOGLE_CLOUD_PROJECT` is required** for the real client ?
`GoogleCloudRunJobClient.run_job`

```

```

156     fails fast with a clear `RuntimeError` if it is unset/empty (it must resolve a real
`job_path`;
157     an unset project would otherwise build a bogus `projects/None/...` name). Export it
(step 3) before
158     dispatching. The guard runs before the `google-cloud-run` import, so it is exercised
in CI even
159     though the SDK isn't a CI/test dependency.
160     - **Hand-rolled `jobs execute --args` with `gs://` URIs:** gcloud's default comma-split
plus its
161     alternate-delimiter syntax both bite ? repeated `--set` tokens trip the comma parser,
and a
162     `^:^` delimiter splits the `:` inside `gs://`. Use a delimiter that appears nowhere in
the args,
163     e.g. `--args='^@@^-v@@infer@@--video-uri=gs://?'`. (The dispatcher path doesn't have
this
164     problem ? prefer it; this is for one-off diagnostics.) Container-arg overrides must
include the
165     dispatcher's two forced `--set`s: `deployment.compute_target=local_gpu`,
166     `indexing.detector_impl=native`.
167     - **Git Bash (MSYS) on Windows mangles Unix-looking values** before gcloud sees them:
168     `--add-volume-mount ?mount-path=gcs` became `C:/Program Files/Git/gcs`, and
169     `--command=/bin/bash` became `C:/Program Files/Git/usr/bin/bash` (the container then
exec-fails).
170     Run gcloud commands with such flags from PowerShell (`MSYS_NO_PATHCONV=1` breaks
gcloud's own
171     wrapper). PowerShell has its own trap: quote any flag value containing commas.
172     - **First execution after an image push** spends minutes in "Container image import in
progress"
173     (visible in `executions describe status.conditions`, NOT in logs) ? that's progress,
not a hang;
174     later executions reuse the imported image.
175

```

13. assistant (2026-07-07T14:51:40.042Z)

README \$2a already carries a deploy-together warning for #507 and claims a new-control-plane-to-old-worker argparse fail (rc 2). Let me check how orchestration.py actually forwards decode_backend ? via `--set` or a `--decode-backend` flag.

14. user (2026-07-07T14:51:41.695Z)

```

520         f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
521         False,
522     )
523     # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
524     # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
525     # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
526     req.video_path = staged_uri
527     input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
528     db.add_video(
529         video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
530     )
531     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
532     if not out_root:
533         return _failed(
534             "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
535             "? that's where the Cloud Run job writes the result.",
536             False,
537         )
538

```

```

539     notify("dispatching (cloud_run)")
540     logger.info(
541         "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
542         req.video_path,
543         out_root,
544     )
545     # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
546     # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
547     # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
548     # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
549     overrides = []
550     idx_cfg = getattr(cfg, "indexing", None)
551     sk = getattr(idx_cfg, "skip_ai_handoff", None)
552     if sk is not None:
553         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
554     # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
555     # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
556     # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
557     # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
558     # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
559     # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
560     wasb_cfg = getattr(idx_cfg, "wasb", None)
561     if wasb_cfg is not None:
562         decode_backend = getattr(wasb_cfg, "decode_backend", None)
563         if decode_backend:
564             overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
565         batch_size = getattr(wasb_cfg, "batch_size", None)
566         if batch_size:
567             overrides.append(f"indexing.wasb.batch_size={batch_size}")
568         prefetch = getattr(wasb_cfg, "prefetch_batches", None)
569         if prefetch:
570             overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
571     # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
572     # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
573     # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
574     # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
575     # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
576     # then worker-rejected at its default 20.0 after a successful GPU dispatch).
577     spec = ComputeJobSpec(
578         video_uri=req.video_path,
579         out_uri=out_root,
580         segmenter=seg_name,
581         config_overrides=tuple(overrides),
582         skip_validation=True,
583     )
584     try:
585         # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
586         # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
587         # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which

```



```

588         # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
589         result = compute.run_infer(
590             spec,
591             on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
592         )
593     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
594         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
595         return _failed(f"cloud dispatch error: {e}", True)
596     if result.returncode != 0:
597         return _failed(f"cloud job failed (returncode {result.returncode}).", True)
598     if result.video_id and result.video_id != video_id:
599         # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
600         # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
601         logger.warning(
602             "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
603             "and the worker hash; ingesting under the local id.",
604             result.video_id,
605             video_id,
606         )
607
608     notify("ingesting (cloud_run)")
609     storage_dict = (
610         storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
611         if hasattr(storage_cfg, "model_dump")
612         else {}
613     )
614     try:
615         _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
616     except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
617         logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
618         return _failed(
619             f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
620             True,
621         )
622
623     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
624     segments = [
625         s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
626     ]
627     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
628     # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
629     # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
630     provenance = {
631         "path": "cloud_run",
632         "requested": req.compute,
633         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
634         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
635         "execution": result.execution,
636         "outputs_uri": result.outputs_uri,
637     }
638     logger.info(
639         "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",

```

The code forwards `decode\_backend` via the pre-existing `--set` arg (line 564), NOT a dedicated argparse flag ? confirming `--skip-validation` is genuinely the first NEW argparse flag on the dispatch path. Let me verify the ExecutionPolicy default (30-min poll) and the rc mapping, plus confirm no wrapper catches SystemExit.

16. user (2026-07-07T14:52:08.734Z)

```
=== DEFAULT_POLICY / max_wait_sec ===
8:  ``max_wait_sec`` deadline).
42:  """An op hung past ``op_timeout_sec`` or never became ready within ``max_wait_sec``."""
46:class ExecutionPolicy:
54:    max_wait_sec: float = 1800.0 # total wall budget for a poll loop
75:DEFAULT_POLICY = ExecutionPolicy()
85:    policy: ExecutionPolicy = DEFAULT_POLICY,
154:    policy: ExecutionPolicy = DEFAULT_POLICY,
163:    ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
177:        policy.max_wait_sec,
187:        if elapsed >= policy.max_wait_sec:
192:            f"{label} not ready within max_wait_sec={policy.max_wait_sec}s"
```

17. user (2026-07-07T14:52:16.320Z)

```
=== get_compute_backend / policy wiring ===
backend/compute/__init__.py:3:COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is
the seam: it returns a
backend/compute/__init__.py:23:    "get_compute_backend",
backend/compute/__init__.py:36:def get_compute_backend(
backend/compute/__init__.py:83:    return CloudRunCompute(
backend/compute/__init__.py:89:        policy=policy,
backend/compute/cloud_run.py:98:            "to resolve the job path; set it on the control
plane (see get_compute_backend)."
```

```
backend/compute/cloud_run.py:149:class CloudRunCompute(ComputeBackend):
backend/orchestration.py:409:    (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for
every non-``cloud_run`` target, so
backend/orchestration.py:411:    from backend.compute import ComputeJobSpec, get_compute_backend
backend/orchestration.py:457:        compute = get_compute_backend(cfg,
target_override="cloud_run")
backend/orchestration.py:459:        compute = get_compute_backend(
```

18. user (2026-07-07T14:52:20.708Z)

```
55     """
56     target = (
57         target_override
58         if target_override is not None
59         else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
60     )
61     if target != "cloud_run":
62         return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
63
64     # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
65     from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
66
67     # Derive the poll budget from config UNLESS the caller injected an explicit policy ?
68     # deployment.remote_poll_max_wait_sec must outlast the worker Job's --task-timeout
so a
69     # slow-but-healthy paid GPU run is never abandoned mid-flight (CP-rebuild $4.3; the
old
70     # 1800s default undercut real 1080p60 jobs). Identity check on DEFAULT_POLICY keeps
an
71     # explicitly-passed policy (tests, callers with bespoke budgets) untouched.
72     if policy is DEFAULT_POLICY:
```

```

73         max_wait = getattr(
74             getattr(config, "deployment", None), "remote_poll_max_wait_sec", None
75         )
76         if max_wait:
77             policy = dataclasses.replace(DEFAULT_POLICY, max_wait_sec=float(max_wait))
78
79         job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
80         region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
81         project = os.environ.get("GOOGLE_CLOUD_PROJECT")
82         client = run_client if run_client is not None else GoogleCloudRunJobClient()
83         return CloudRunCompute(
84             run_client=client,
85             job_name=job_name,
86             region=region,
87             project=project,
88             storage_config=_storage_dict(config),
89             policy=policy,
90             token=token,
91         )
92

```

19. user (2026-07-07T14:52:29.080Z)

```

backend/api/job_worker.py:388:     ``deployment.remote_poll_max_wait_sec`` budget a live dispatch
gets), then complete or
backend/api/job_worker.py:432:         getattr(getattr(config, "deployment", None),
"remote_poll_max_wait_sec", 3900.0)
Binary file backend/api/__pycache__/job_worker.cpython-313.pyc matches
backend/compute/__init__.py:68:     # deployment.remote_poll_max_wait_sec must outlast the worker
Job's --task-timeout so a
backend/compute/__init__.py:74:         getattr(config, "deployment", None),
"remote_poll_max_wait_sec", None
Binary file backend/compute/__pycache__/__init__.cpython-313.pyc matches
backend/config/models.py:483:     remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
Binary file backend/config/__pycache__/models.cpython-313.pyc matches

```

20. assistant (2026-07-07T14:52:50.802Z)

Let me confirm whether the worker has a `--decode-backend` argparse flag at all, and re-verify the exact new flag location and parse ordering.

21. user (2026-07-07T14:52:53.558Z)

```

=== any --decode-backend argparse flag? ===
=== all add_argument in infer subparser (context) ===
702:     pi.add_argument(
710:         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
711:         pi.add_argument(
714:         pi.add_argument(
719:         pi.add_argument(
722:         pi.add_argument(
725:         pi.add_argument(
728:         pi.add_argument("--max-frames", type=int, default=None)
729:         pi.add_argument(
736:         pi.add_argument(
743:         pi.add_argument(
751:         pi.set_defaults(func=cmd_infer)
753:         pt = sub.add_parser(
757:         pt.add_argument(
763:         pt.add_argument(
768:         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
769:         pt.add_argument(
777:         pt.add_argument(
782:         pt.add_argument(

```

```

785: pt.add_argument("--config", default=None)
786: pt.add_argument("--scratch", default=None)
787: pt.add_argument(
793: pt.add_argument(
799: pt.add_argument("--set", action="append", default=[], metavar="k=v")
800: pt.set_defaults(func=cmd_train)
822: args = build_parser().parse_args(argv)

```

22. assistant (2026-07-07T14:54:04.390Z)

Confirmed: there is NO `--decode-backend` argparse flag ? `decode_backend` is forwarded via the pre-existing `--set` (argparse-safe), so `--skip-validation` is genuinely the first NEW argparse flag on the dispatch path. Let me check how `--set` overrides are applied on the worker (to assess whether a `--set`-based alternative would degrade more gracefully than a hard argparse abort), and verify the #510 runlog claim about the control-plane being redeployed alone.

23. user (2026-07-07T14:54:08.367Z)

```

=== how --set overrides are applied in cmd_infer ===
61: def _apply_overrides(config: Any, sets: List[str]) -> None:
62:     """Apply `a.b.c=value` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
188:     config_overrides=tuple(args.set or ()),
236:     _apply_overrides(config, args.set)
540:     _apply_overrides(config, args.set)
=== #510 / control-plane redeploy runlog ===
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_alpha-all1v1_2026-07-06.md

```

24. user (2026-07-07T14:54:21.903Z)

```

61     def _apply_overrides(config: Any, sets: List[str]) -> None:
62         """Apply `a.b.c=value` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
63         for kv in sets or []:
64             key, sep, val = kv.partition("=")
65             if not sep:
66                 raise ValueError(f"--set expects k=v, got {kv!r}")
67             parts = key.split(".")
68             obj = config
69             for p in parts[:-1]:
70                 obj = getattr(obj, p)
71             setattr(obj, parts[-1], _coerce(val))
72
73
74     def _write_intervals_csv(segments: List[dict], path: str) -> None:
75         """Decimal-second `[start,end)` rallies + shot count / ending reason ? the join-
friendly
76         artifact (same schema as the rally-annotator labels)."""
77         with open(path, "w", newline="") as f:
78             w = csv.writer(f)
79             w.writerow(["start", "end", "shot_count", "ending_reason"])
80             for s in segments:
81                 w.writerow(
82                     [
83                         f"{float(s.get('start_time', 0.0)):.3f}",
84                         f"{float(s.get('end_time', 0.0)):.3f}",
85                         s.get("shot_count", ""),
86                         s.get("ending_reason", s.get("shot_count_confidence", "")),
87                     ]
88                 )
89
90
91     def _write_report_json(
92         video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""

```

```

93     ) -> None:
94         with open(path, "w") as f:
95             json.dump(
96                 {
97                     "video_id": video_id,
98                     "status": status,
99                     "segment_count": len(segments),
100                     # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
101                     # control plane re-hydrating from outputs/<md5>/ restores the video row
102                     # with its truthful path ? without this, only the segments recover.
103                     "video_uri": video_uri,
104                     "segments": [
105                         {
106                             "start": float(s.get("start_time", 0.0)),
107                             "end": float(s.get("end_time", 0.0)),
108                         }
109                         for s in segments
110                     ],

```

25. user (2026-07-07T14:54:30.232Z)

```

225         local = os.path.join(scratch, "_done.json")
226         with open(local, "w", encoding="utf-8") as f:
227             json.dump(marker, f)
228         storage.put(local, done_uri, config=storage_cfg)
229         logger.info("[worker] wrote completion marker -> %s", done_uri)
230     except Exception as e: # never fail a good run because the marker push hiccuped
231         logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
232
233
234     def cmd_infer(args: argparse.Namespace) -> int:
235         config = load_config(args.config) if args.config else load_config()
236         _apply_overrides(config, args.set)
237         if not _run_startup_checks(config):
238             return 2
239
240         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
241         # process.
242         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
243         # through to the
244         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
245         # (the
246         # dispatcher forces compute_target=local_gpu), so it never recursively re-
247         # dispatches.
248         compute = get_compute_backend(config)
249         if compute is not None:
250             return _dispatch_remote(compute, args)
251
252         storage_cfg = config.storage.model_dump()
253
254         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
255         os.makedirs(scratch, exist_ok=True)
256         config.output.output_dir = scratch
257
258         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
259         # to scratch.
260         local_video = storage.fetch(
261             args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
262         )
263         print(f"[worker] pulled {args.video_uri} -> {local_video}")
264
265         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
266         # byte-identical).
267         video_id = compute_video_id(local_video)

```

```
263         # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
264         # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
```

26. assistant (2026-07-07T14:55:12.217Z)

Let me verify the ComputeJobSpec call sites (the finding's self-declared "non-issue") to make sure there's no hidden defect there, and confirm the frozen dataclass field ordering is safe.

27. user (2026-07-07T14:55:14.575Z)

```
=== ComputeJobSpec construction call sites ===
backend/orchestration.py:577:     spec = ComputeJobSpec(
backend/worker/__main__.py:184:     spec = ComputeJobSpec(
tests/test_compute_backend.py:184:         ComputeJobSpec(
tests/test_compute_backend.py:209:         ComputeJobSpec(
tests/test_compute_backend.py:237:         ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri=str(tmp_path / "bk"))
tests/test_compute_backend.py:240:         ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri=str(tmp_path / "bk"))
tests/test_compute_backend.py:308:         ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri="gs://b")
tests/test_compute_backend.py:335:             ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri=str(tmp_path / "bucket"))
tests/test_compute_backend.py:357:             ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri=str(tmp_path / "bucket"))
tests/test_compute_backend.py:394:         ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri="gs://b")
tests/test_compute_backend.py:780:         ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri=str(tmp_path / "bk")),
```

28. assistant (2026-07-07T14:55:40.516Z)

I now have everything needed to render a verdict. Let me confirm the key facts one more time by reasoning through them.

All call sites use kwargs with ``skip_validation`` as a trailing ``bool = False`` field ? the frozen-dataclass change is benign (confirmed non-issue).

The finding's core mechanics all check out against the real code:

1. ****New argparse flag, unconditional emission****: ``--skip-validation`` is a genuinely new ``add_argument`` (worker ``__main__.py:743``), consumed by strict ``parse_args`` (``:822``). ``orchestration.py:582`` sets ``skip_validation=True`` on ****every**** dispatch; ``cloud_run.py:199-202`` appends ``--skip-validation`` to the container argv. Confirmed ``decode_backend`` (#507) is forwarded via the pre-existing ``--set`` (line 564) ? there is ****no**** ``--decode-backend`` argparse flag ? so this is indeed the first NEW argparse flag on the dispatch path.

2. ****Old worker aborts before the marker****: ``main()`` calls ``build_parser().parse_args(argv)`` (``:822``) ? argparse ``SystemExit(2)`` on the unknown flag, ****before**** ``args.func(args)`` runs. The done-marker is written only inside ``cmd_infer`` (success path / ``_fail`` at ``:312-315``). ENTRYPOINT is bare ``python3 -m backend.worker`` (Dockerfile:123) with ``sys.exit(main())`` ? nothing catches ``SystemExit``. So: rc 2, no marker.

3. ****Dispatcher only bucket-polls the marker****: ``run_infer`` polls ``_read_marker(done_uri)`` (``cloud_run.py:217``), never the Cloud Run API status. No marker ? ``poll_until`` runs the full budget ? ``PolicyTimeout`` ? ``_handle_poll_timeout`` (final absent check, cancel, re-raise) ? ``orchestration.py:593-595`` surfaces a generic ``cloud dispatch error``.

One correction that ****strengthens**** the finding: the production poll budget is ``deployment.remote_poll_max_wait_sec``, default ****3900s** (~65 min)** (``config/models.py:483``), not 1800s. The 1800s is only the ``DEFAULT_POLICY`` fallback, which ``get_compute_backend`` overrides from config. So the hang is ~65 min, longer than the finding states.

The README §2a deploy-together note exists only for #507's decode_backend (lines 76-78); no equivalent note was added for this change.

29. user (2026-07-07T14:56:05.212Z)

Structured output provided successfully